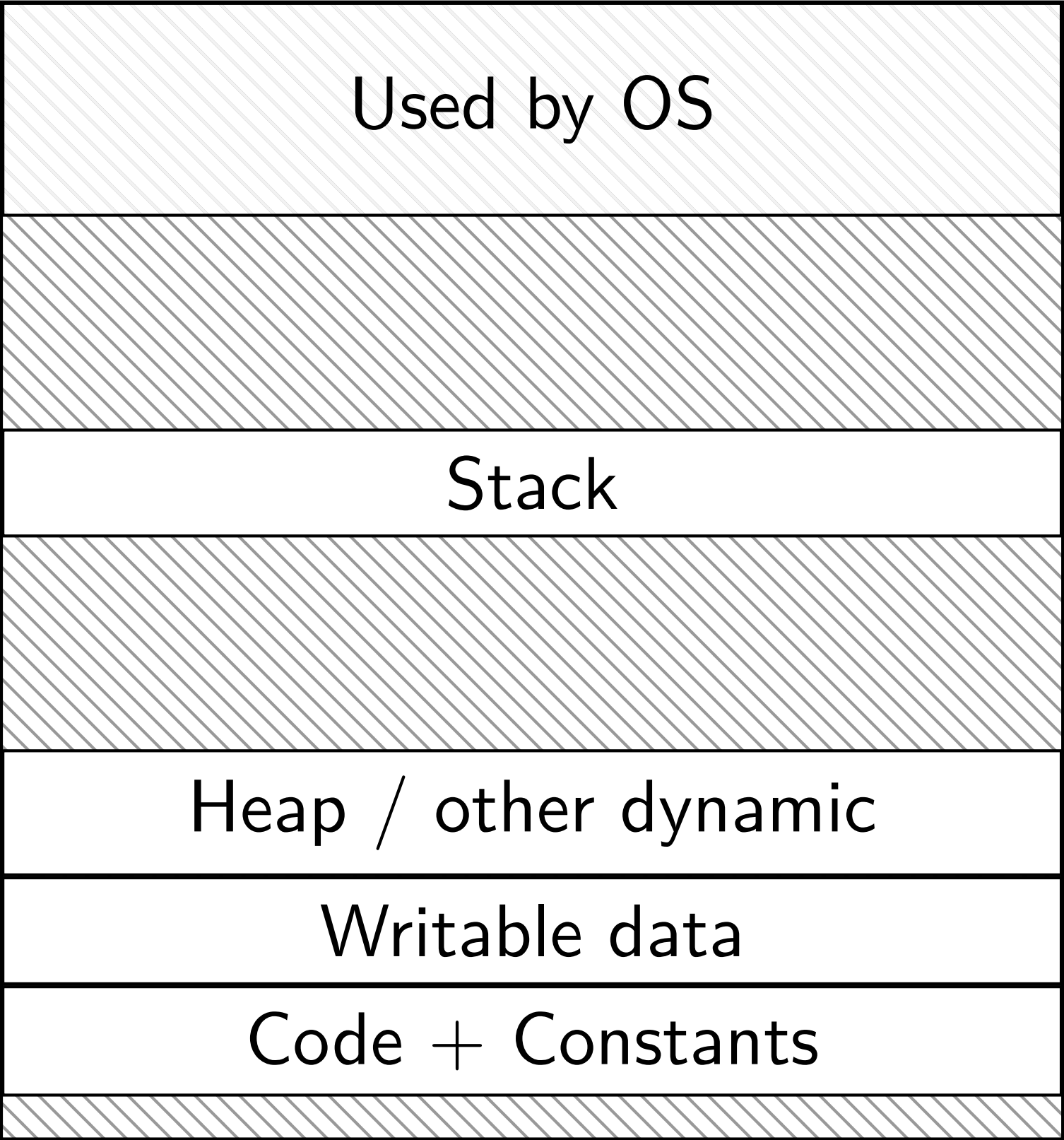


program memory



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

0x7F...

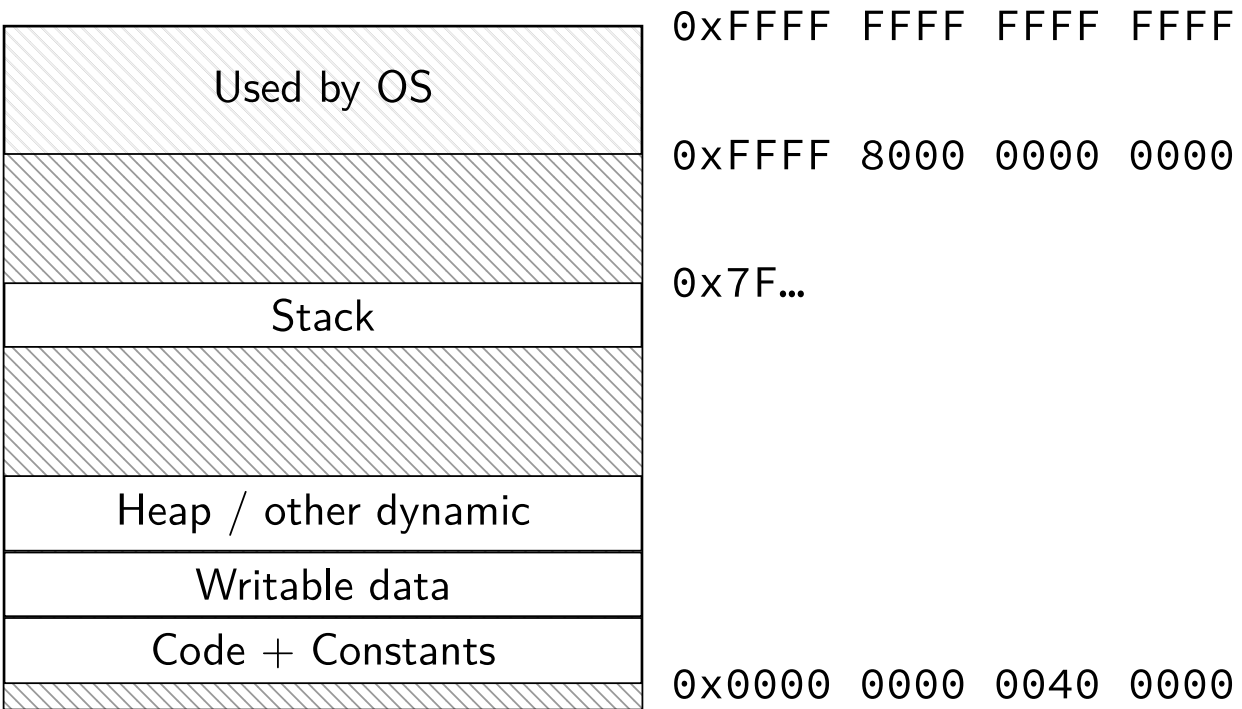
0x0000 0000 0040 0000

this layout is what we want

programs know their addresses when compiled
compiler can generate smaller, more efficient code

programs have (essentially) unlimited memory
or at least it appears that way

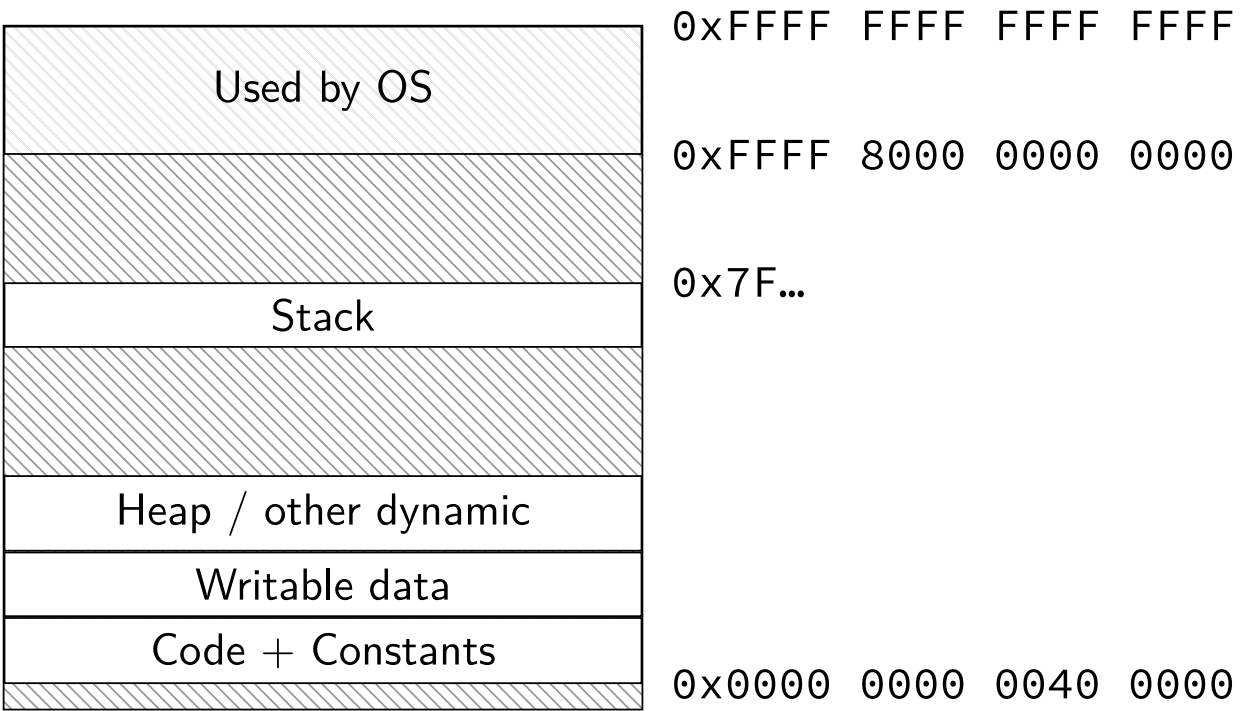
programs are not machine-specific (within same ISA)
i.e., don't need to know how much memory is installed



but, this has a number of issues!

every program uses the same addresses
could only run one program at a time

our machines don't actually have unlimited memory



virtualization allows us to address this

virtualization: allow many users to access the same computing hardware resources as if they were the only user

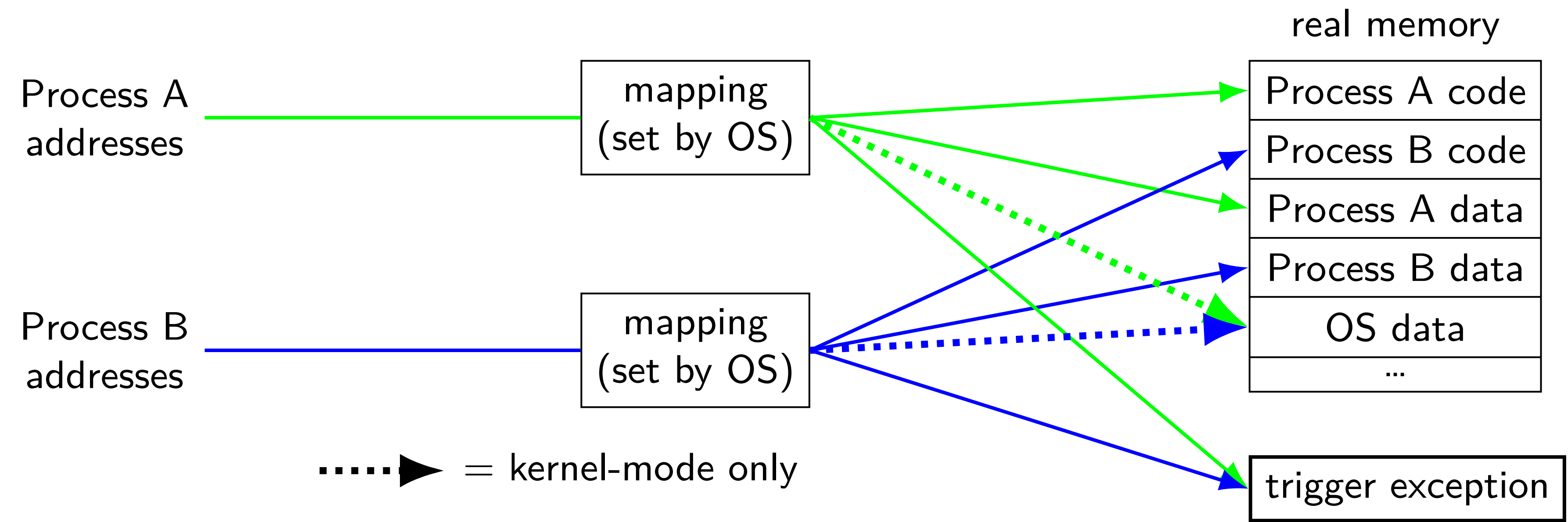
examples:

- virtual machine: multiple OSes running on same hardware

- virtual memory: multiple processes using same physical memory

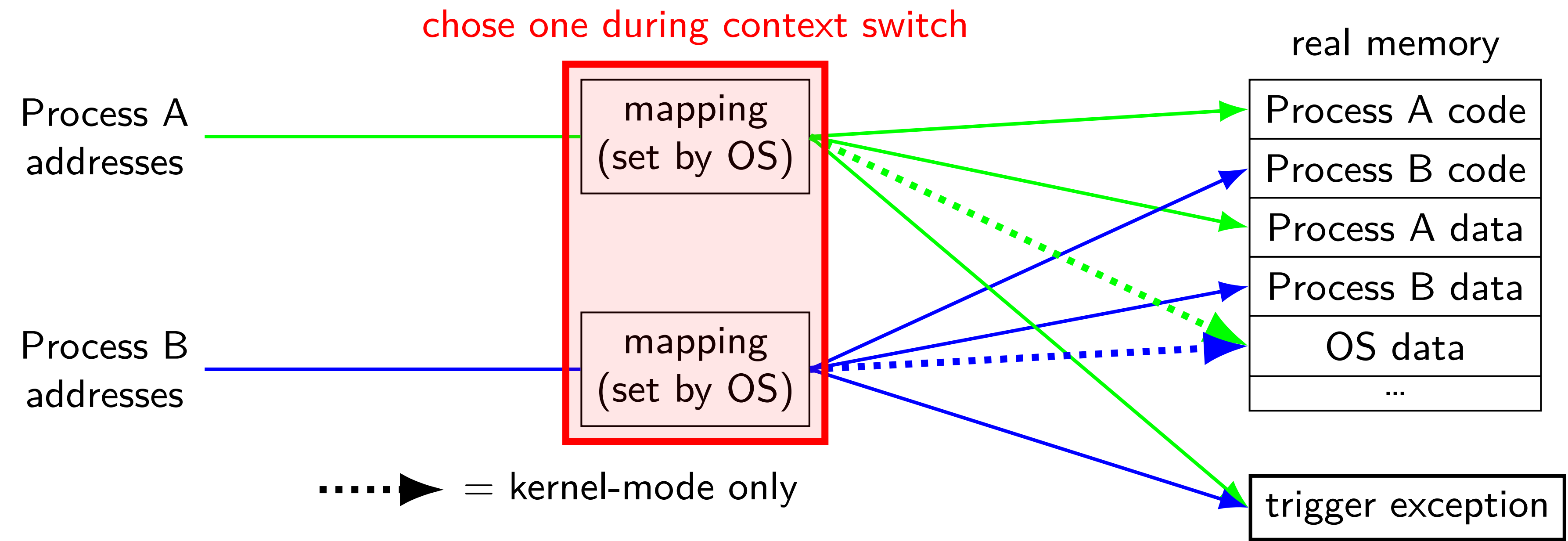
address spaces

illusion of *dedicated memory*

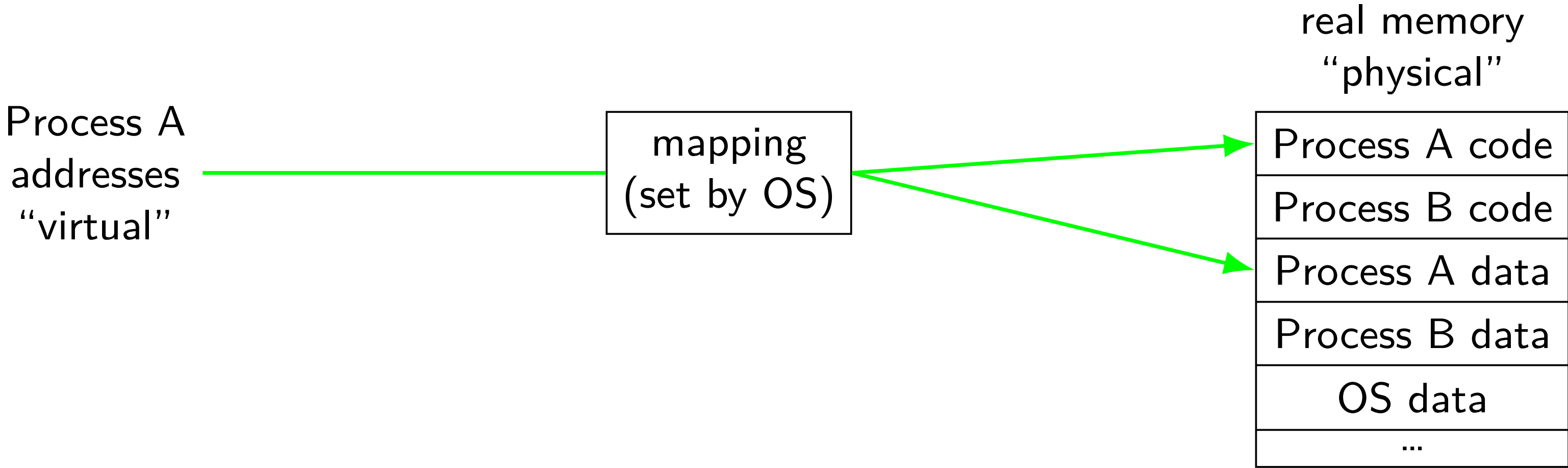


address spaces

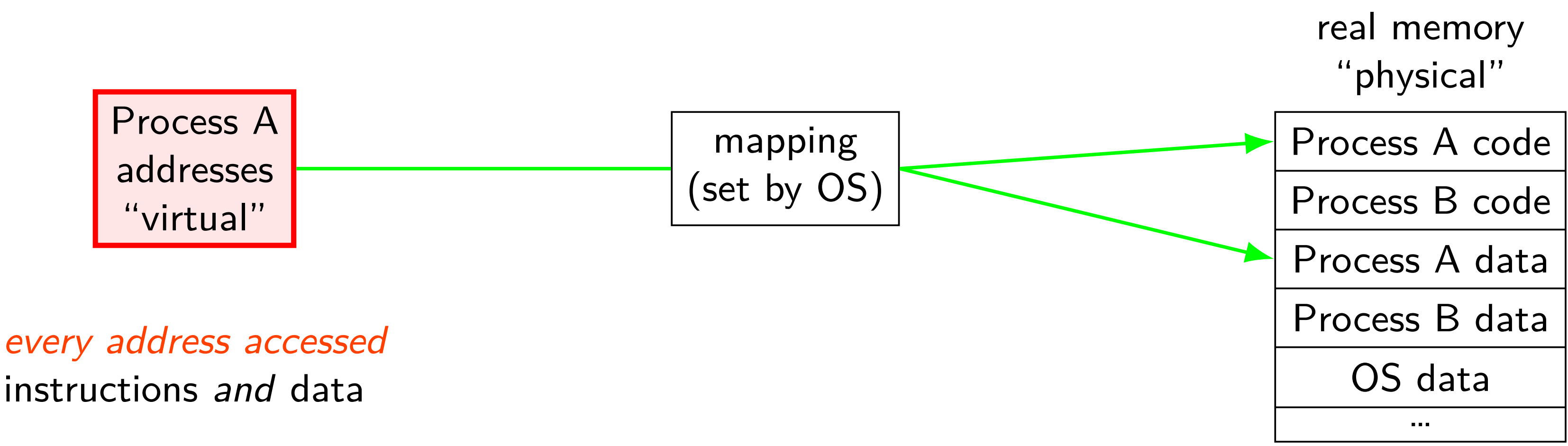
illusion of *dedicated memory*



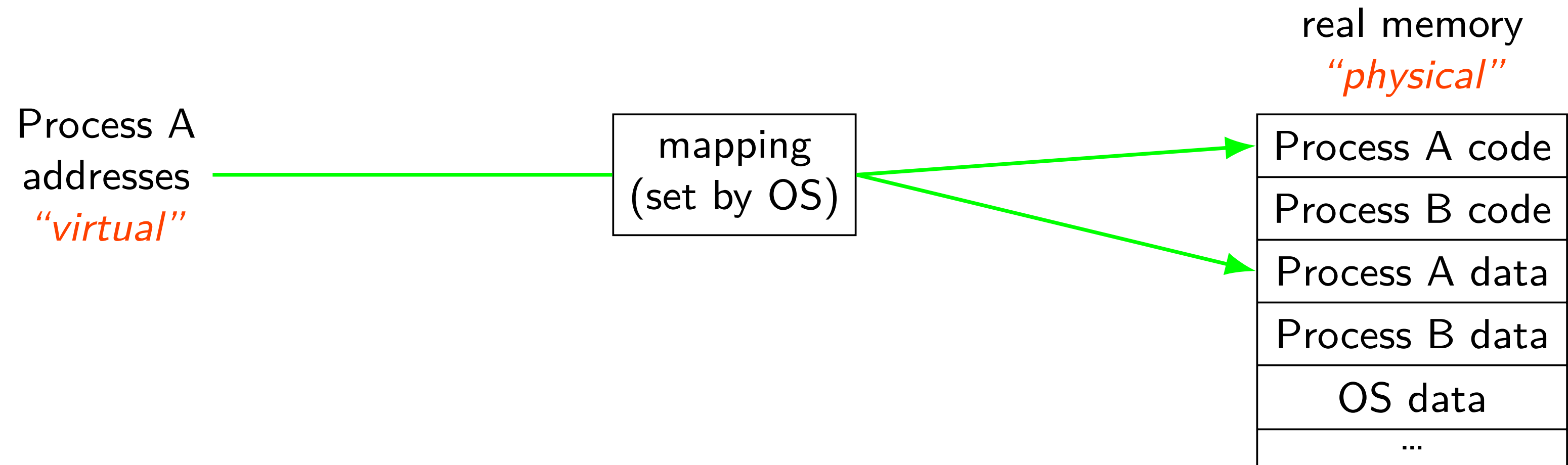
address translation



address translation

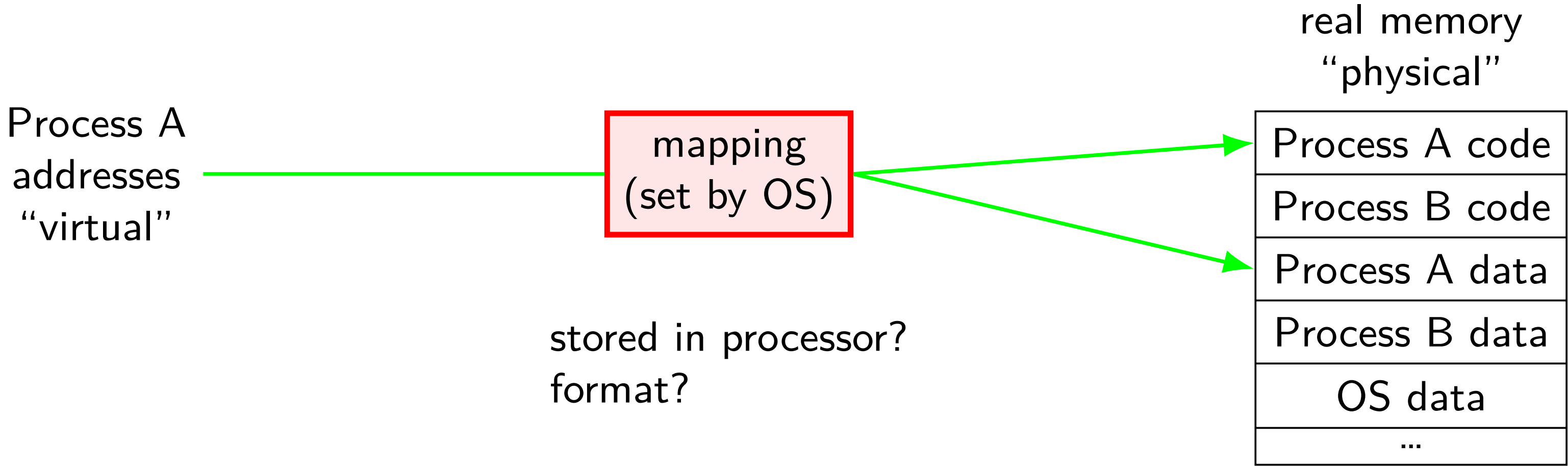


address translation

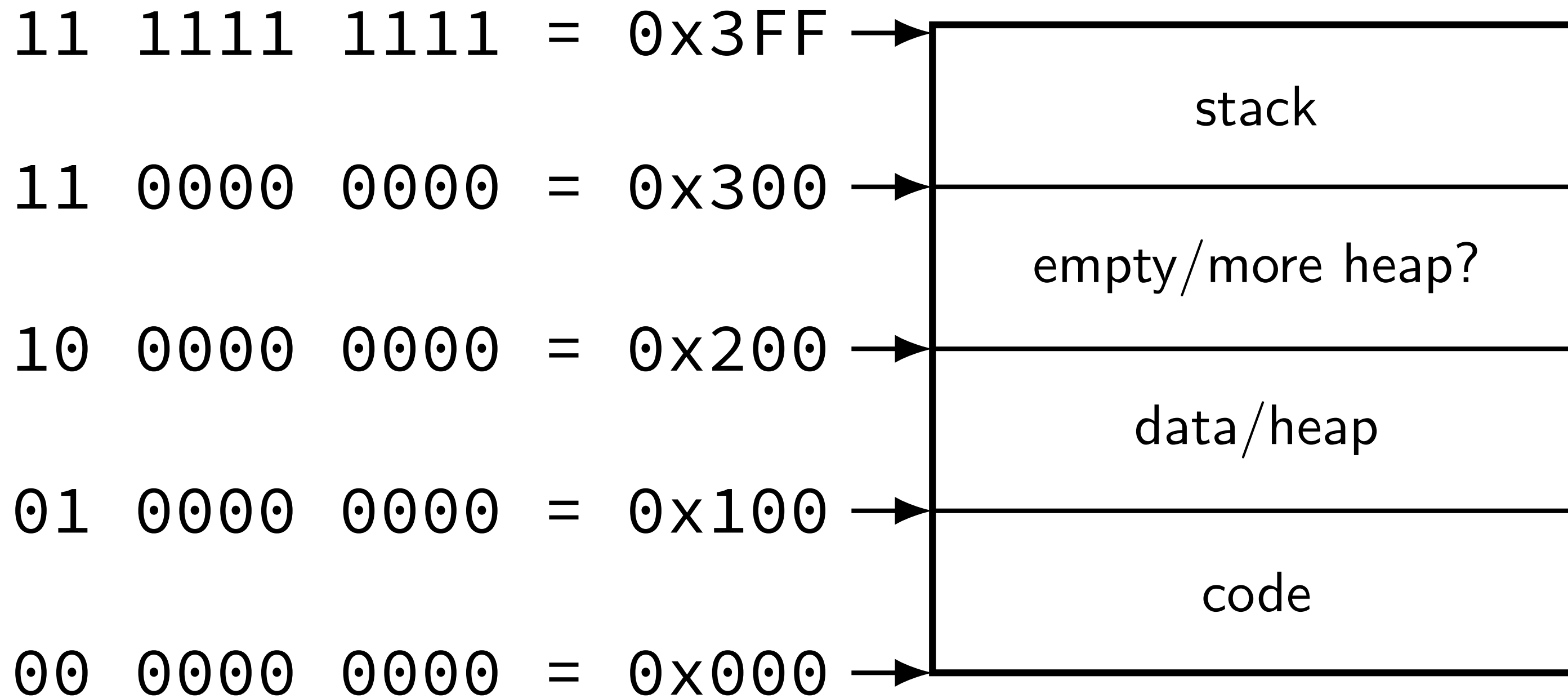


program addresses are 'virtual'
real addresses are 'physical'
can be *different sizes*!

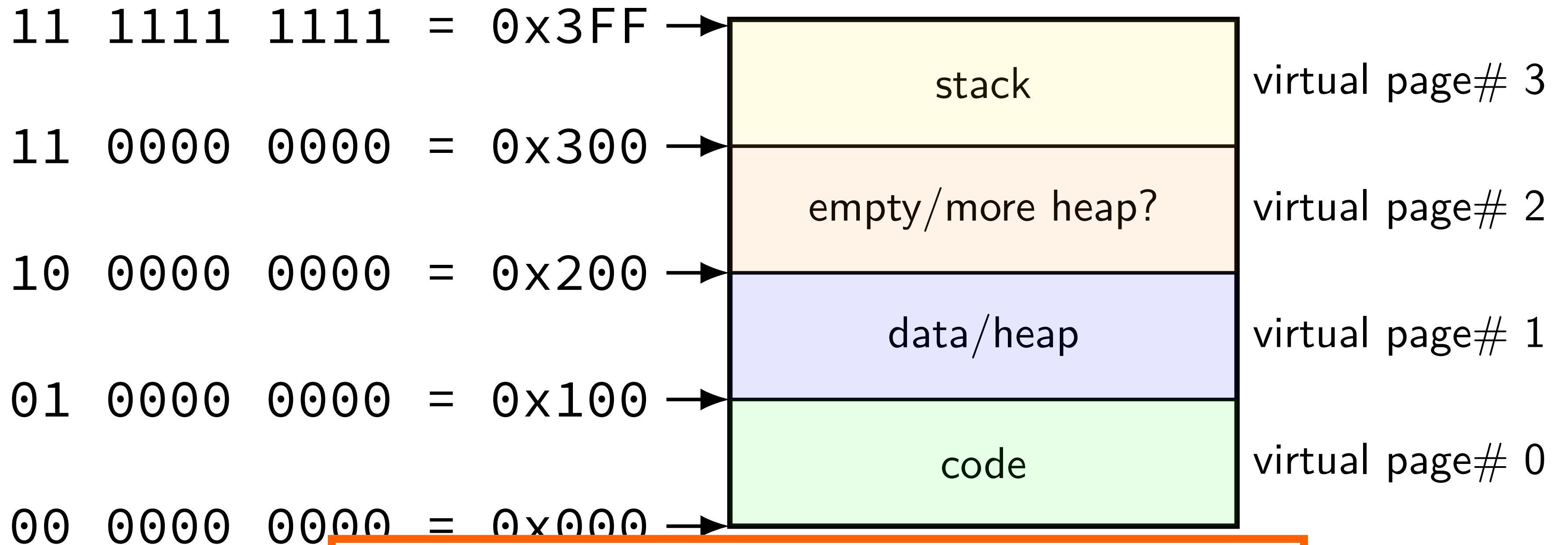
address translation



toy program memory

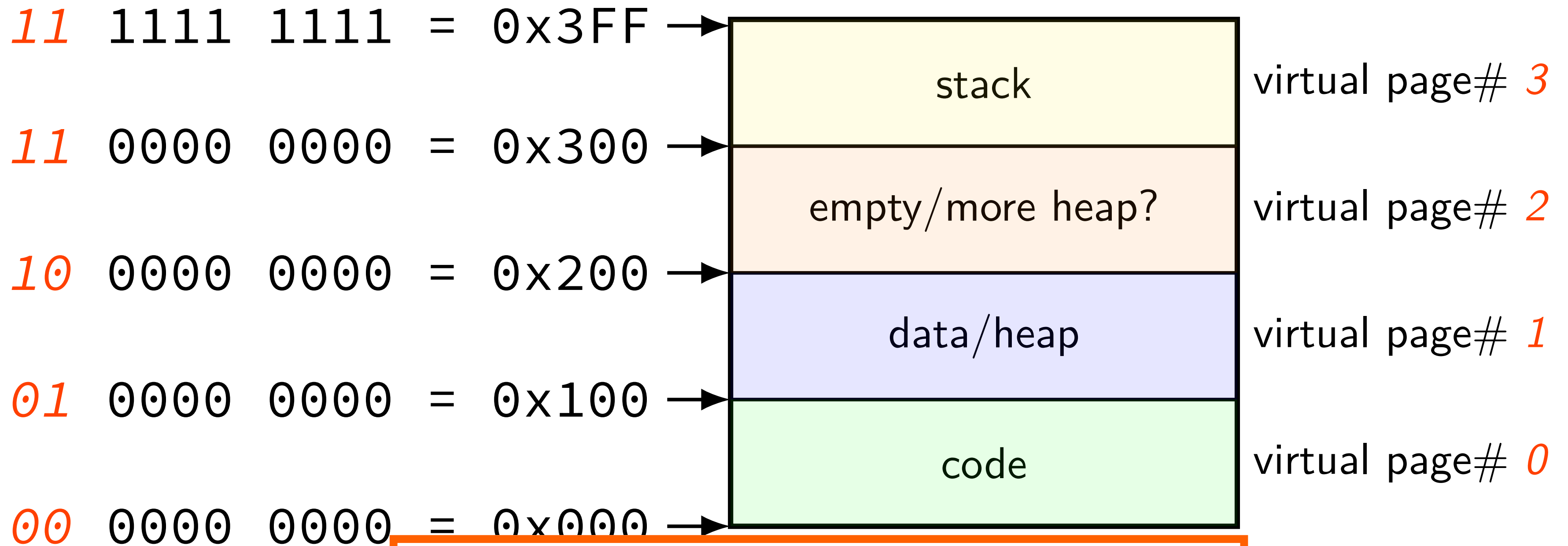


toy program memory



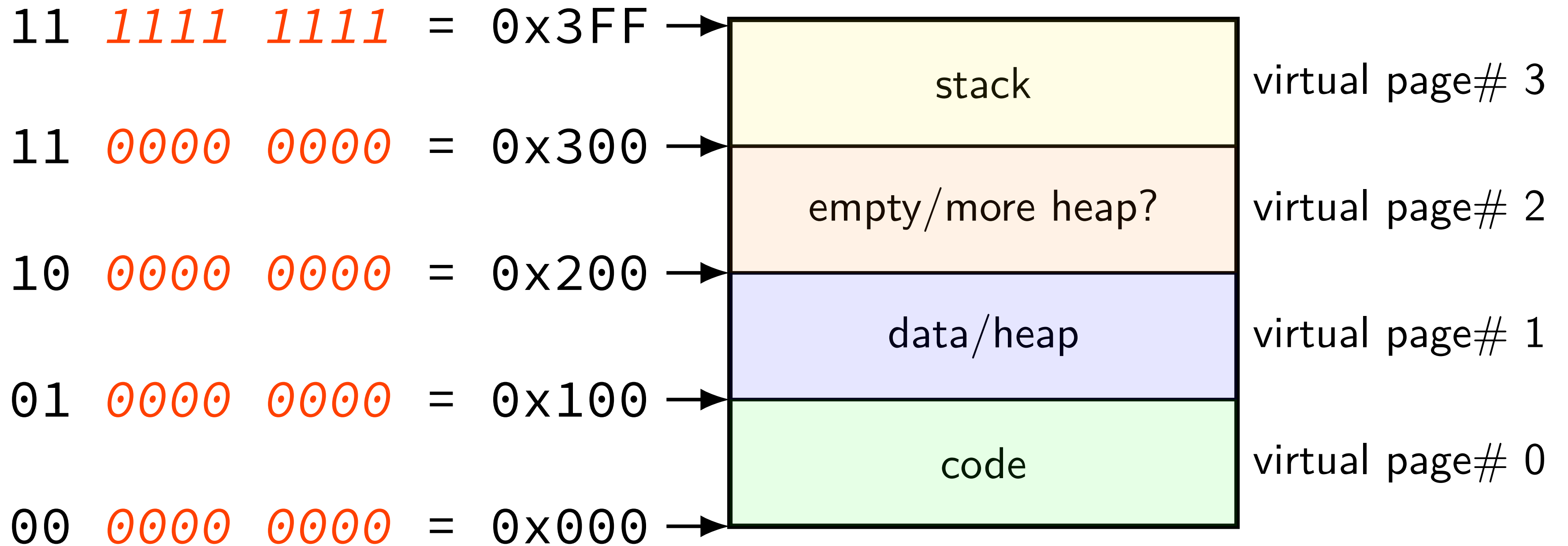
divide memory into *pages* (2^8 bytes in this case)
“virtual” = addresses the program sees

toy program memory



page number is the upper bits of address
because the page size is a power of two

toy program memory



the rest of the address is called the *page offset*

toy physical memory

toy physical memory

program memory
virtual addresses

11	0000	0000	to
11	1111	1111	
10	0000	0000	to
10	1111	1111	
01	0000	0000	to
01	1111	1111	
00	0000	0000	to
00	1111	1111	

real memory
physical addresses

111	0000	0000	to
111	1111	1111	
001	0000	0000	to
001	1111	1111	
000	0000	0000	to
000	1111	1111	

toy physical memory

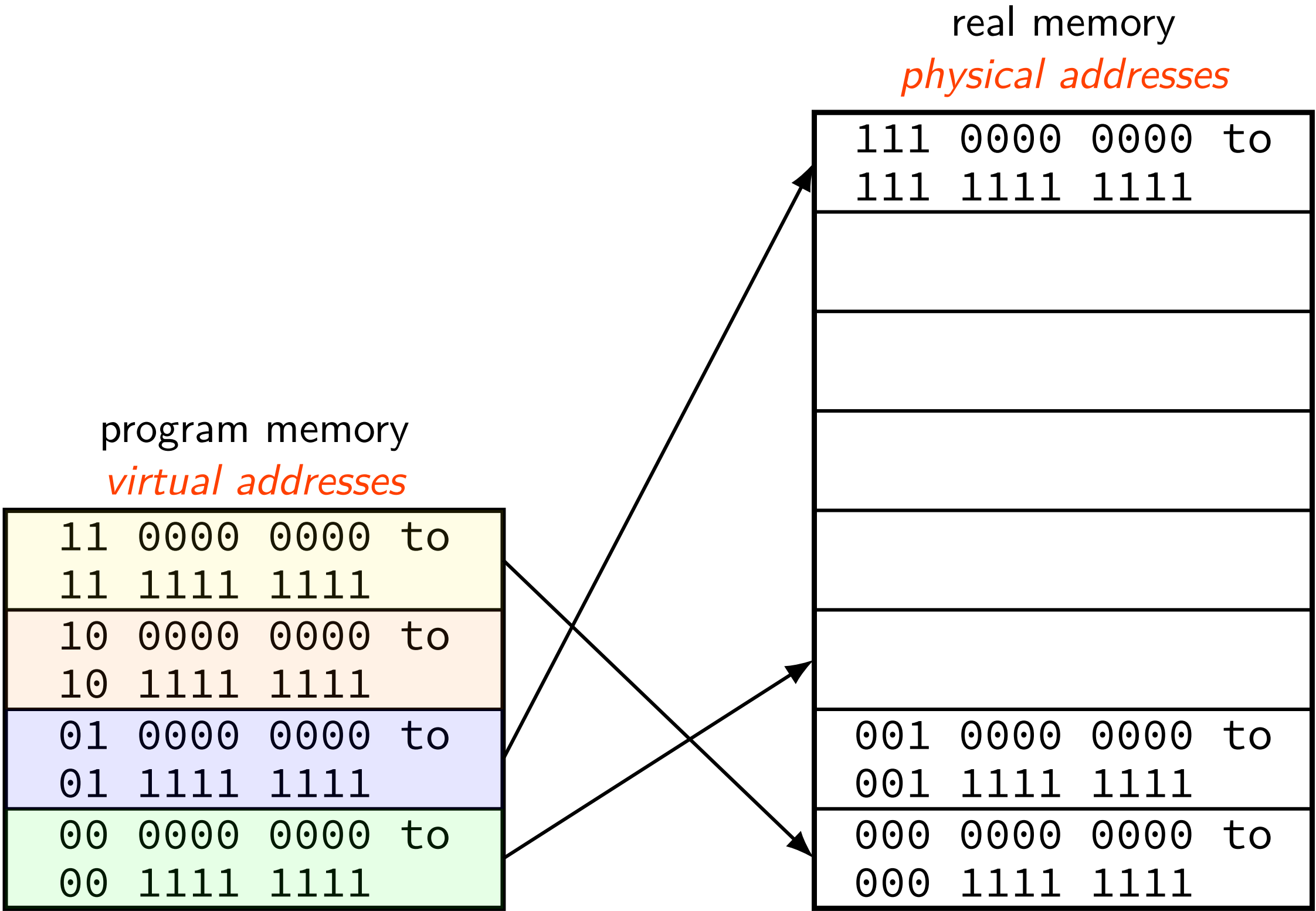
program memory
virtual addresses

11	0000	0000	to
11	1111	1111	
10	0000	0000	to
10	1111	1111	
01	0000	0000	to
01	1111	1111	
00	0000	0000	to
00	1111	1111	

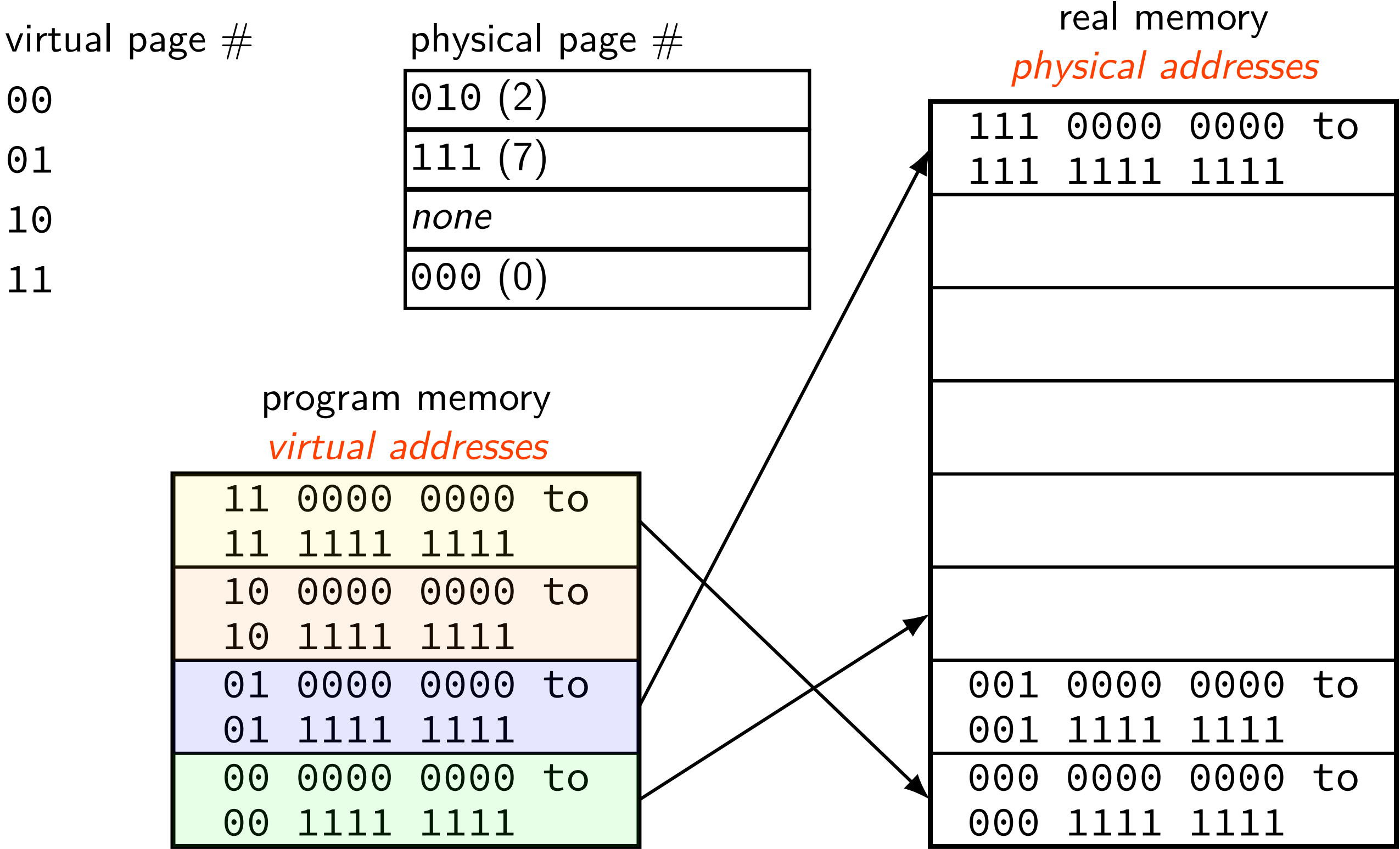
real memory
physical addresses

111	0000	0000	to	physical page 7
111	1111	1111		
001	0000	0000	to	physical page 1
001	1111	1111		
000	0000	0000	to	physical page 0
000	1111	1111		

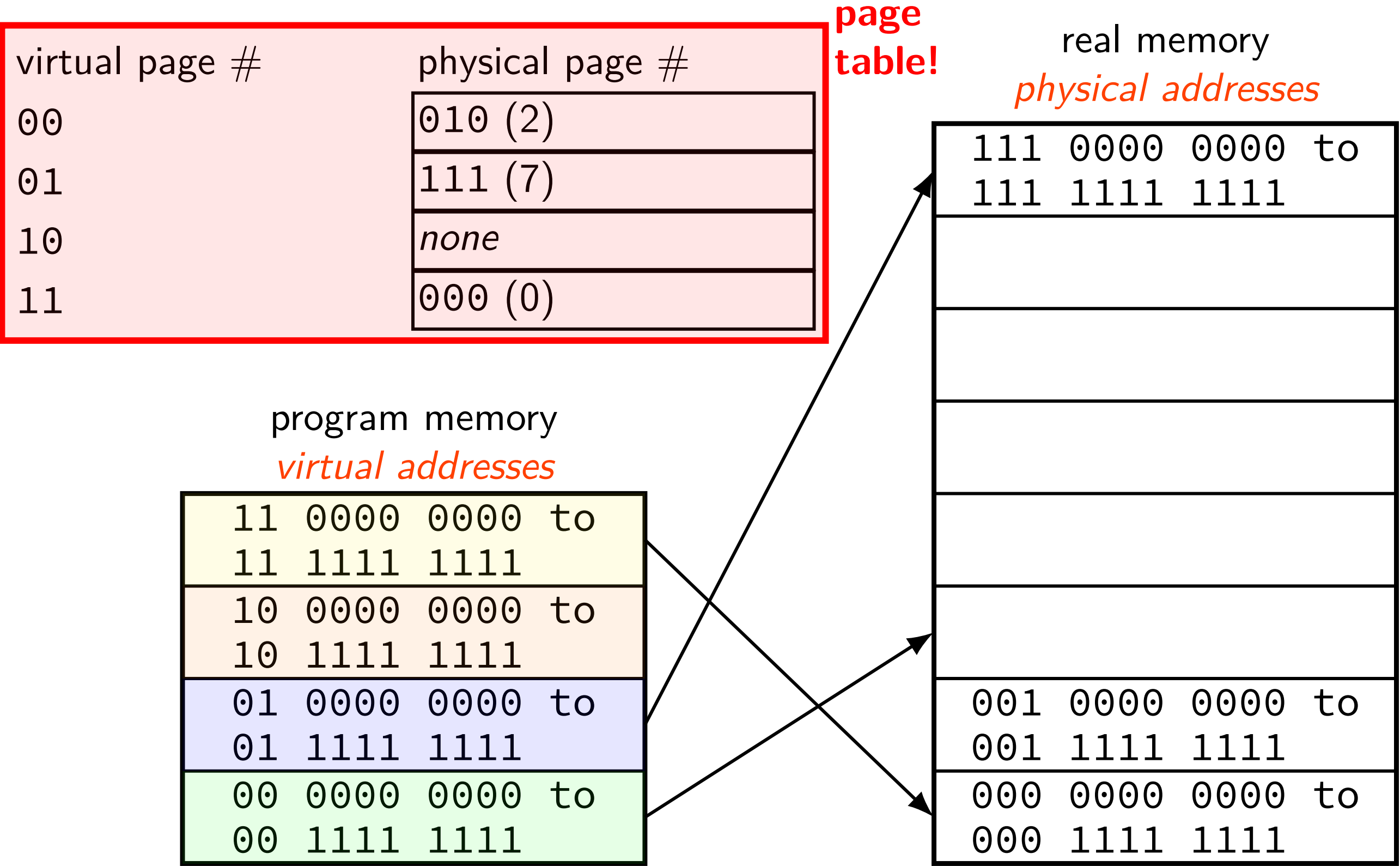
toy physical memory



toy physical memory



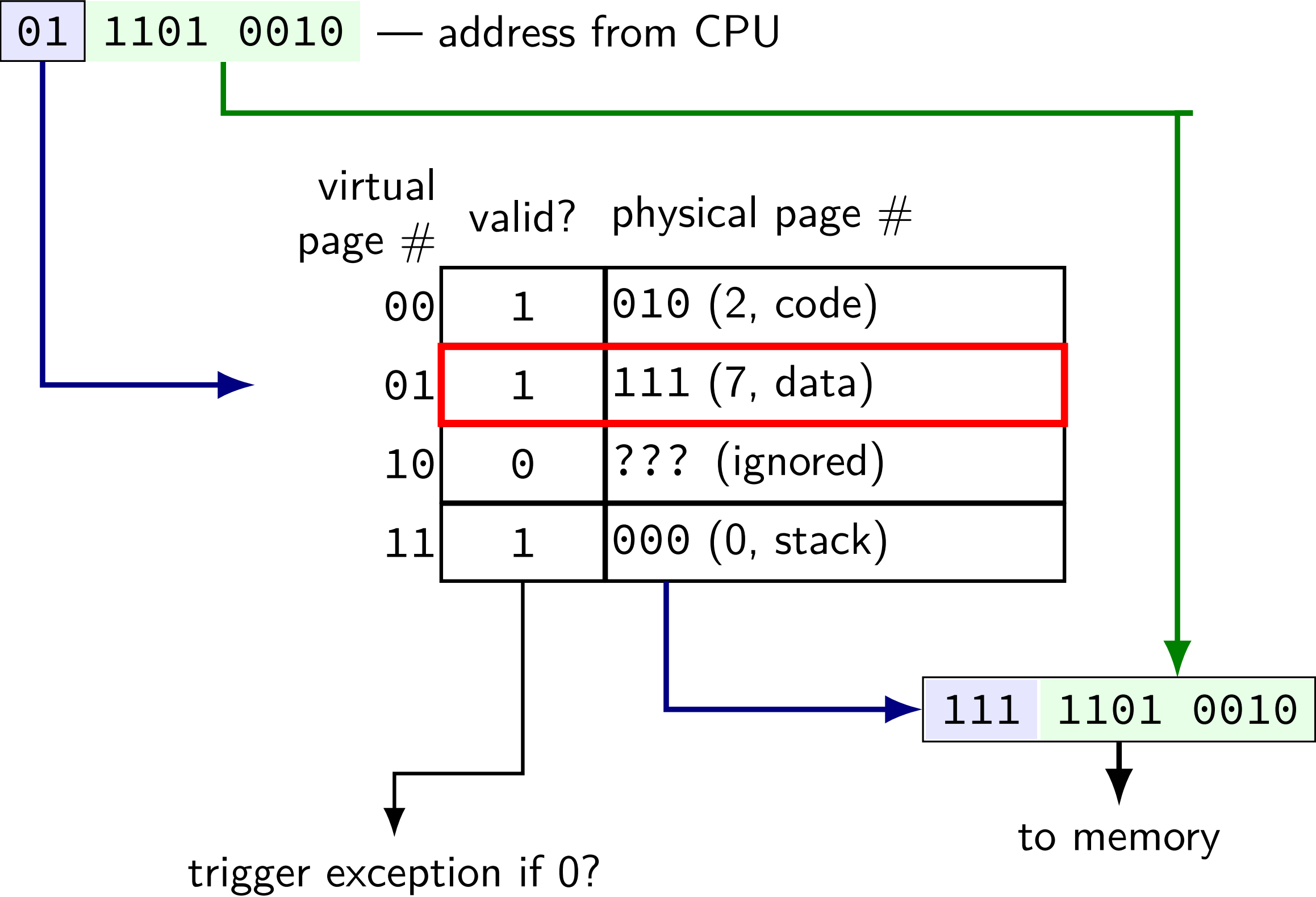
toy physical memory



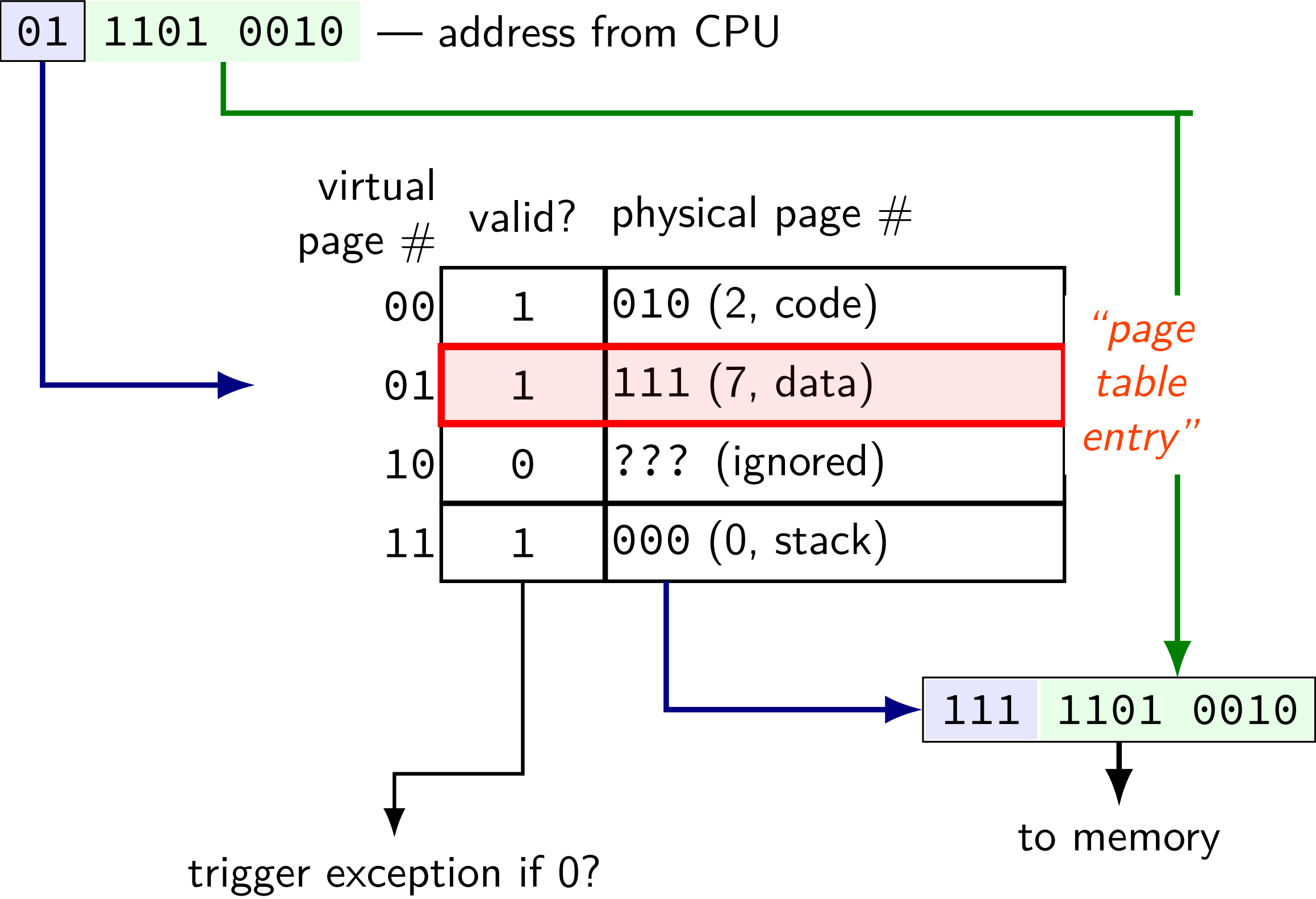
toy page table lookup

virtual page #	valid?	physical page #
00	1	010 (2, code)
01	1	111 (7, data)
10	0	??? (ignored)
11	1	000 (0, stack)

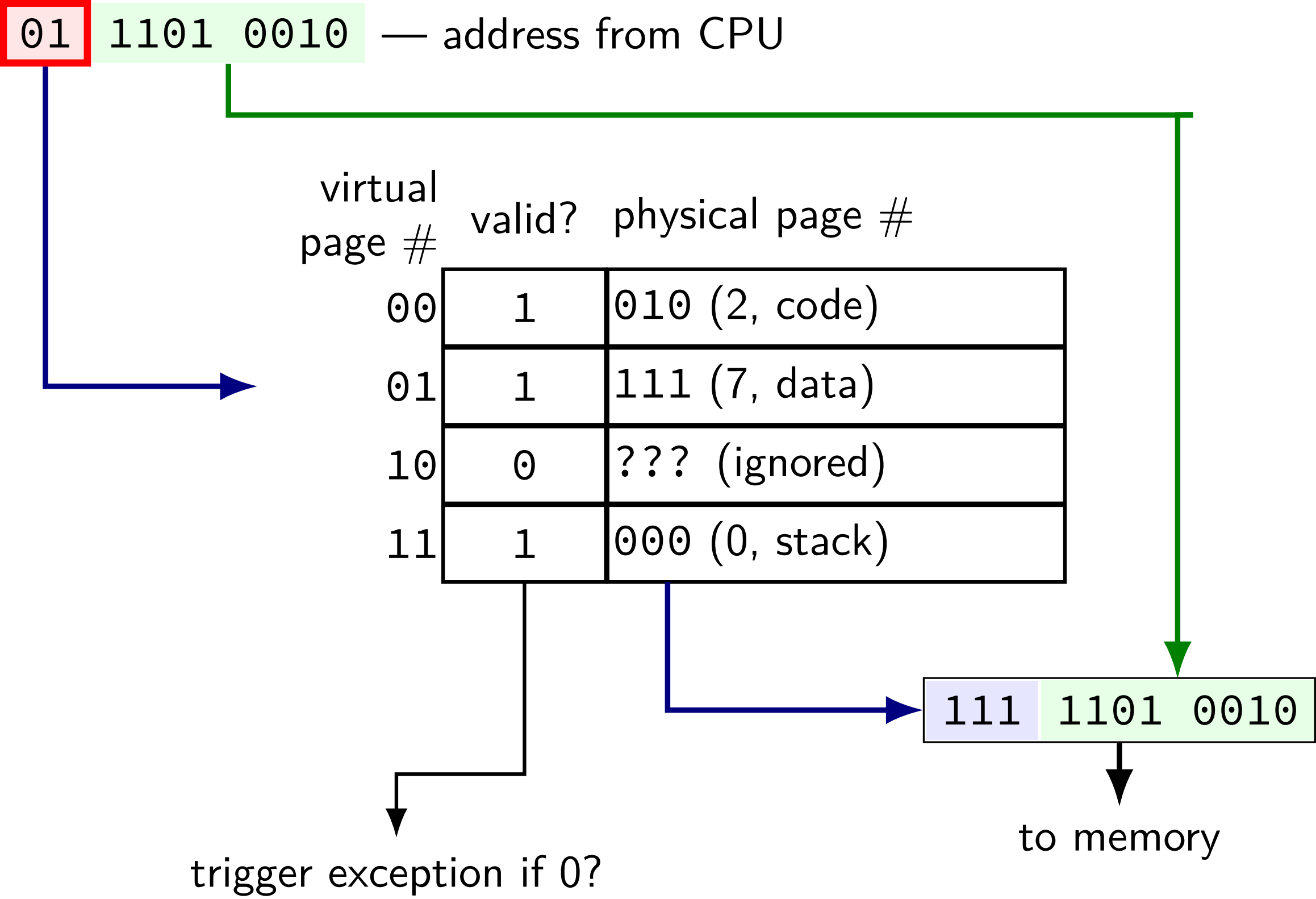
toy page table lookup



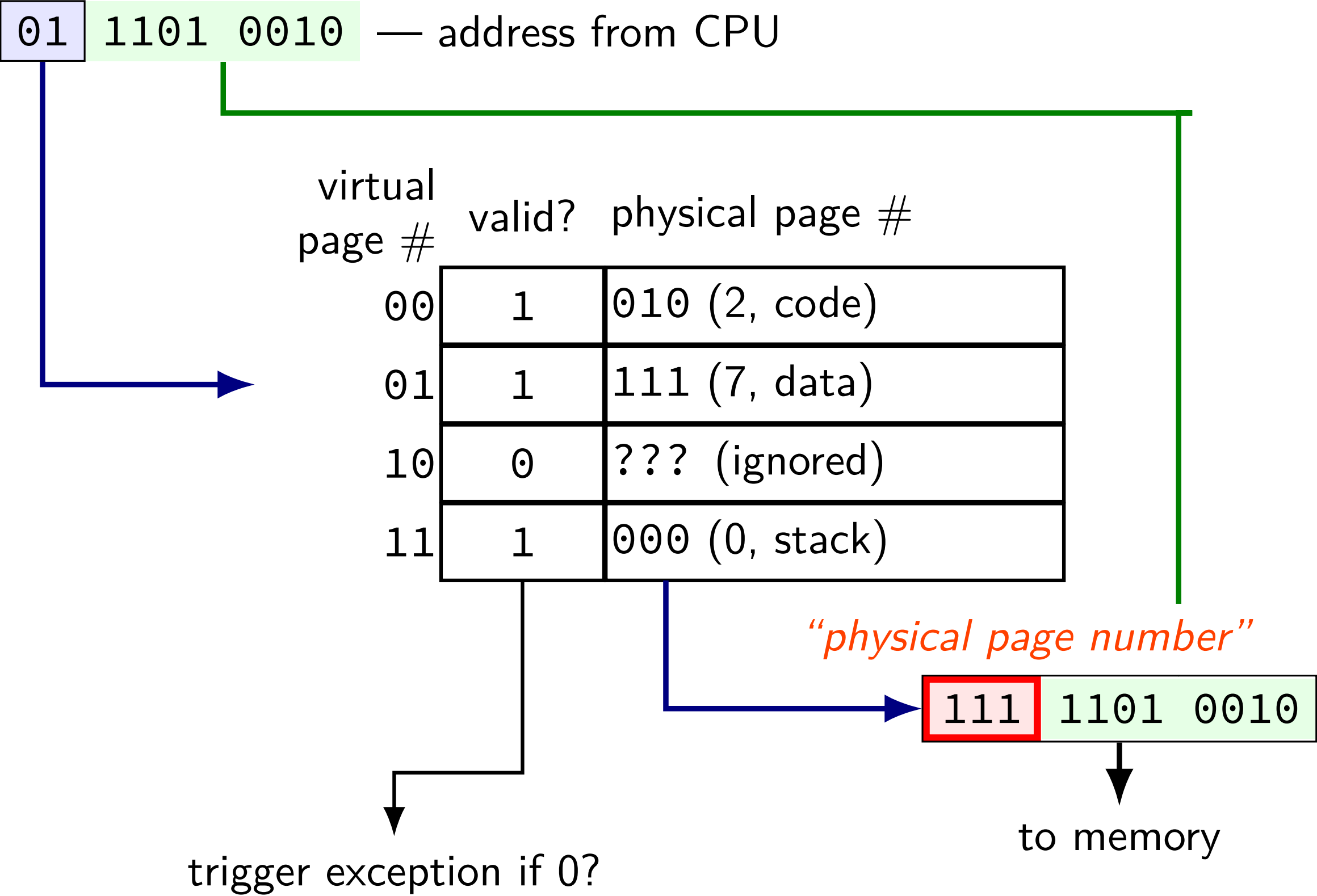
toy page table lookup



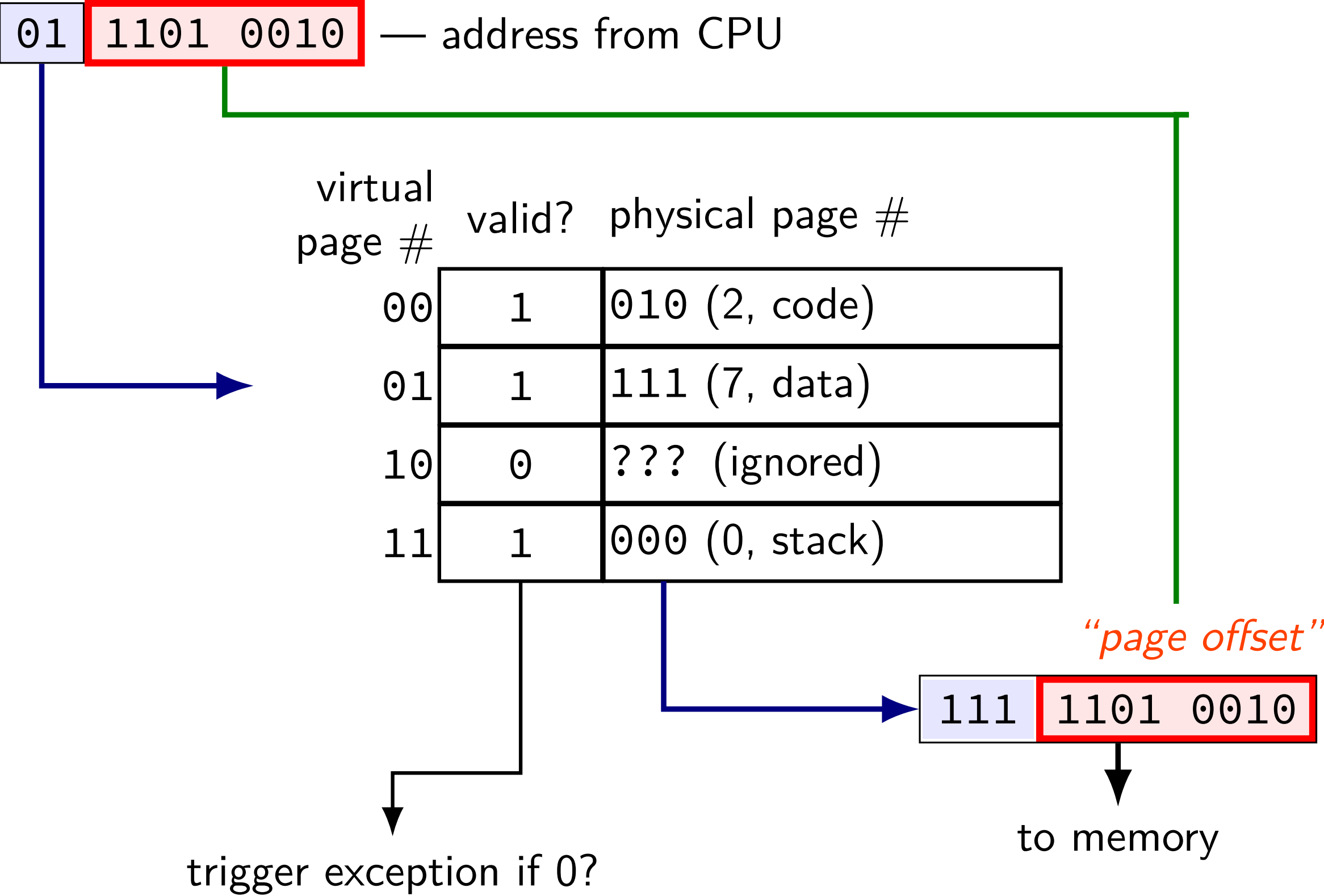
toy page table lookup



toy page table lookup



toy page table lookup



on virtual address sizes

virtual address size = size of pointer?

often, but — sometimes part of pointer not used

example: typical x86-64 only use 48 bits

rest of bits have fixed value

virtual address size is amount used for mapping

address space sizes

amount of stuff that can be addressed = address space size
based on number of unique addresses

e.g. 32-bit virtual address = 2^{32} byte virtual address space

e.g. 20-bit physical addressss = 2^{20} byte physical address space

what if my machine has 3GB of memory (not power of two)?

not all addresses in physical address space are useful

most common situation (since CPUs support having a lot of memory)

exercise: page counting

suppose 32-bit virtual (program) addresses
and each page is 4096 bytes (2^{12} bytes)

how many virtual pages?

exercise: page counting

suppose 32-bit virtual (program) addresses
and each page is 4096 bytes (2^{12} bytes)

how many virtual pages?

$$2^{32} / 2^{12} = 2^{20}$$

exercise: page table size

suppose 32-bit virtual (program) addresses

suppose 30-bit physical (hardware) addresses

each page is 4096 bytes (2^{12} bytes)

page table entries have physical page #, valid bit

how big is the page table (if laid out like ones we've seen)?

exercise: page table size

suppose 32-bit virtual (program) addresses

suppose 30-bit physical (hardware) addresses

each page is 4096 bytes (2^{12} bytes)

page table entries have physical page #, valid bit

how big is the page table (if laid out like ones we've seen)?

2^{20} entries $\times$ (18 + 1) bits per entry

exercise: page table size

suppose 32-bit virtual (program) addresses

suppose 30-bit physical (hardware) addresses

each page is 4096 bytes (2^{12} bytes)

page table entries have physical page #, valid bit

how big is the page table (if laid out like ones we've seen)?

2^{20} entries $\times$ (18 + 1) bits per entry

19922944 bits = 2490368 bytes

exercise: page table size

suppose 32-bit virtual (program) addresses

suppose 30-bit physical (hardware) addresses

each page is 4096 bytes (2^{12} bytes)

page table entries have physical page #, valid bit

how big is the page table (if laid out like ones we've seen)?

2^{20} entries $\times$ (18 + 1) bits per entry

19922944 bits = 2490368 bytes

issue: where can we store that?

exercise: address splitting

if each page is 4096 bytes (2^{12} bytes)

split the address `0x12345678` into page number and page offset:

exercise: address splitting

if each page is 4096 bytes (2^{12} bytes)

split the address 0x12345678 into page number and page offset:

page number: 0x12345; offset: 0x678

exercise: page table lookup

suppose 64-byte pages (= 6-bit page offsets), 9-bit virtual addresses

VPN	valid	PPN
000	1	0010
001	1	1010
010	0	—
011	0	—
100	1	1110
101	1	0100
110	1	0001
111	0	—

virtual address 0x024 (0 0010 0100) = physical address ???

backup slides

exercise

40-bit physical addresses, 36-bit virtual addresses, 2^{14} byte (16384 byte) pages

Q1: number of virtual pages?

Q2: size of physical page numbers?

exercise: page table lookup (2)

suppose 32-byte pages (= 5-bit page offsets), 8-bit virtual addresses

VPN	valid	PPN
000	1	10
001	1	01
010	0	—
011	0	—
100	1	00
101	0	—
110	0	—
111	0	—

virtual address 0x97 = ???